《Web 前端交互与事件监听·复习专用总结》

一、Fetch 与 Axios：写法对比与核心区别

在试卷中，如果是“代码填空”或“编写代码片段”，一定要看清题目要求是用原生 API (Fetch) 还是第三方库 (Axios)。

1. 核心区别对比表

特性	Fetch (原生)	Axios (第三方库)	考试记忆点
环境	浏览器内置，无需安装	需要引入 JS 文件或 npm 安装	题目若未提引入库，首选 Fetch
数据解析	**两步走**：先拿响应流，再 `.json()`	**自动**：直接在 `.data` 中获取 JSON	Fetch 必须写 `return res.json()`
POST 数据	手动转字符串 `JSON.stringify(data)`	自动转换 JSON	Fetch 容易忘写 `JSON.stringify`
Headers	POST 时需手动设置 `Content-Type`	自动识别（大部分情况）	Fetch 必须写 `application/json`
错误处理	只有网络断开才报错，404/500 不会进 catch	状态码非 2xx 会直接报错	Axios 错误处理更符合直觉

2. 代码编写模板（必背）

场景 A：GET 请求（获取数据）

- **Fetch 写法：**

  JavaScript

  ```
  fetch('/api/users?id=1')
    .then(response => response.json()) // 第一步：解析响应头，返回 Promise
    .then(data => {
        console.log(data); // 第二步：拿到真正的 JSON 数据
    })
    .catch(err => console.error(err));
  ```

- **Axios 写法：**

  JavaScript

  ```
  axios.get('/api/users?id=1')
    .then(response => {
        console.log(response.data); // 直接拿 data
    })
    .catch(err => console.error(err));
  ```

场景 B：POST 请求（提交数据）

- **Fetch 写法（易错点多，重点记）：**

  JavaScript

  ```
  fetch('/api/users', {
      method: 'POST',
      headers: {
          'Content-Type': 'application/json' // 必须写！否则后端收不到 JSON
      },
      body: JSON.stringify({ name: 'Tom', age: 18 }) // 必须手动转字符串！
  })
  .then(res => res.json())
  .then(data => console.log(data));
  ```

- **Axios 写法：**

  JavaScript

  ```
  axios.post('/api/users', {
      name: 'Tom',
      age: 18
  }) // 自动转 JSON，自动加 Header，很爽
  .then(res => {
      console.log(res.data);
  });
  ```

二、Web 通信技术原理：除了 AJAX 还有什么？

AJAX 是最基础的，但不是唯一的。考试可能会问“实时聊天怎么实现”，这时不能只答 AJAX。

1. AJAX (Asynchronous JavaScript and XML)

- **原理**：浏览器通过 `XMLHttpRequest` 对象（或 `Fetch` API）向服务器发送异步请求。**“客户端主动请求，服务器被动响应”**。
- **适用场景**：
  - 搜索框自动提示（用户输入触发）。
  - 点击“加载更多”新闻列表。
  - 表单提交验证。
- **缺点**：服务器不能主动发消息给客户端。如果想做实时消息，只能用**轮询**（前端每隔几秒问一次服务器“有新消息吗”），效率低。

2. WebSocket (全双工通信)

- **原理**：基于 TCP 的持久化协议。握手成功后，建立一条专门的通道，**“客户端和服务器可以随时互相发消息”**。
- **适用场景**：
  - **在线即时聊天**（微信网页版）。
  - **多人在线游戏**（王者荣耀）。
  - **协同文档编辑**（腾讯文档、Google Docs）。

    - **股票实时K线图。**
- **代码特征**：通常看到 `new WebSocket('ws://...')`。

3. SSE (Server-Sent Events，服务器发送事件)

- **原理**：基于 HTTP。建立连接后，**“服务器可以单向一直给客户端发消息，但客户端不能通过这个通道发给服务器”**（单工）。
- **适用场景**：
    - **ChatGPT 的打字机效果**（AI 生成的内容流式传输）。
    - **体育赛事比分直播**（观众不需要发数据给服务器，只看比分）。
    - **系统日志监控。**

三、常用事件监听 (Event Listeners) 详细总结

考试中常考“为了实现XX功能，应该监听什么事件？”。

1. 表单与输入类（Form & Input）

事件名	触发时机	经典应用场景
`oninput`	输入框内容**每变化一次**就触发	**搜索框自动提示**、限制输入字数（如微博 140 字检测）。
`onchange`	内容改变且**失去焦点**（或回车）后触发	**下拉菜单 (`<select>`) 筛选城市**、复选框勾选状态改变。
`onblur`	元素**失去焦点**时触发	**表单验证**（如输入完用户名点别处，检测是否重名）。
`onfocus`	元素**获得焦点**时触发	输入框聚焦时显示下拉建议、显示输入帮助提示。
`onsubmit`	表单**提交**时触发	**拦截传统表单提交**（`e.preventDefault()`），改为 AJAX 提交。

2. 鼠标与交互类（Mouse）

事件名	触发时机	经典应用场景
`onclick`	鼠标**点击**	按钮提交、链接跳转、模态框关闭。
`onmouseover`	鼠标**移入**元素	**显示悬浮菜单**、显示商品详情的大图预览。
`onmouseout`	鼠标**移出**元素	隐藏悬浮菜单。
`onmousemove`	鼠标在元素上**移动**	拖拽验证码、绘图板功能。

3. 键盘类（Keyboard）

事件名	触发时机	经典应用场景
`onkeydown`	按键**按下**（长按会一直触发）	**游戏角色移动**（按住 W 前进）、Ctrl+C 快捷键检测。

事件名	触发时机	经典应用场景
`onkeyup`	按键**抬起**	搜索框按回车键（Enter）触发搜索。

4. 窗口与文档类（Window）

事件名	触发时机	经典应用场景
`onload`	页面资源（图片等）**全部加载完**	初始化地图、绘制 Canvas、统计页面加载时间。
`onscroll`	滚动条**滚动**	**无限滚动加载**（类似抖音/小红书刷到底部加载新内容）、顶部导航栏吸顶效果。
`onresize`	浏览器窗口**大小改变**	响应式布局调整（JS 层面）、Echarts 图表自适应重绘。

四、综合题答题话术示例

如果考试题目问：

> **“请设计一个用户搜索功能，要求用户输入关键词时实时显示建议，点击建议后跳转。”**

解题思路模板：

1. **监听事件**：给输入框 (`<input>`) 绑定 `oninput` 事件（实时性），而不是 `onchange`（需失焦）。
2. **技术原理**：在事件处理函数中，使用 AJAX (Fetch/Axios) 技术向后端发送异步请求。
3. **防抖处理 (Debounce)**（加分项）：为了防止输入一个字就发一次请求导致卡顿，应加入**防抖**逻辑（如：延迟 300ms 再发送请求）。
4. **数据渲染**：后端返回 JSON 数据后，前端使用 DOM 操作（如 `innerHTML`）更新建议列表。

这是非常实用的备考策略。针对期末考试中的**“前端交互 + 后端实现”**综合题，我为您整理了**三个**最经典的场景。

这三种场景涵盖了 Web 开发中 90% 的交互逻辑：**联动加载**、**状态更新**、**分页追加**。请将这些话术和代码直接整理进文档。

场景一：省市二级联动（级联选择）

题目描述：

请设计一个注册页面的地址选择功能。要求：页面有两个下拉框，第一个选择“省份”，第二个选择“城市”。当用户改变“省份”时，“城市”下拉框的内容应自动更新为该省份下的所有城市。

1. 答题话术（设计思路）

> **设计思路：**
>
> 1. **事件监听**：给“省份”下拉框 (`<select>`) 绑定 `onchange` 事件。
> 2. **获取参数**：当事件触发时，获取当前选中的 `provinceId`。
> 3. **数据请求**：使用 AJAX (Fetch/Axios) 向后端发送 GET 请求（如 `/api/cities?provinceId=101`）。

> 4. **DOM 更新**:
    - 清空“城市”下拉框原有的选项。
    - 遍历后端返回的城市列表数据。
    - 动态创建 `<option>` 标签并追加到“城市”下拉框中。

2. 具体实现代码

前端代码 (HTML + JS):

```
<select id="provinceSelect" onchange="loadCities()">
    <option value="1">浙江省</option>
    <option value="2">江苏省</option>
</select>
<select id="citySelect">
    <option>请选择城市</option>
</select>

<script>
function loadCities() {
    const provinceId = document.getElementById('provinceSelect').value;
    const citySelect = document.getElementById('citySelect');

    // 1. 发起请求
    fetch(`/api/cities?provinceId=${provinceId}`)
        .then(res => res.json())
        .then(data => {
            // 2. 清空旧选项 (保留第一个提示项)
            citySelect.innerHTML = '<option>请选择城市</option>';

            // 3. 遍历并渲染新选项
            data.forEach(city => {
                const option = document.createElement('option');
                option.value = city.id;
                option.innerText = city.name;
                citySelect.appendChild(option);
            });
        });
}
</script>
```

后端代码 (Controller + MyBatis):

```
// Controller
@GetMapping("/api/cities")
public List<City> getCitiesByProvince(@RequestParam Long provinceId) {
    return cityMapper.selectByProvinceId(provinceId);
}

// MyBatis XML
// <select id="selectByProvinceId" resultType="City">
//     SELECT * FROM city WHERE province_id = #{provinceId}
// </select>
```

场景二：购物车数量修改（局部状态更新）

题目描述：

设计购物车页面的商品数量修改功能。用户点击“+”按钮时，商品数量加 1，并实时更新数据库；如果更新失败，需提示用户。

1. 答题话术（设计思路）

> **设计思路：**
>
> 1. **事件监听**：给“+”按钮绑定 `onclick` 事件，传入商品 ID。
> 2. **准备数据**：获取当前数量并加 1。
> 3. **数据请求**：使用 Fetch 发送 `PUT` 或 `POST` 请求，将最新的数量发送给后端（如 `/api/cart/update`）。
> 4. **回调处理**：
> - 若请求成功，更新页面上显示的数字，并重新计算总价。
> - 若请求失败（如库存不足），弹出提示框并回滚显示的数字。

2. 具体实现代码

前端代码 (HTML + JS):

```
<span id="count-1001">1</span>
<button onclick="addCount(1001)">+</button>

<script>
function addCount(itemId) {
    // 1. 获取当前数量元素
    const span = document.getElementById(`count-${itemId}`);
    let currentCount = parseInt(span.innerText);

    // 2. 发送请求
    fetch('/api/cart/update', {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ id: itemId, count: currentCount + 1 })
    })
    .then(res => res.json())
```

```
    .then(data => {
        if (data.success) {
            // 3. 更新 UI
            span.innerText = currentCount + 1;
            console.log("更新成功，最新总价：" + data.totalPrice);
        } else {
            alert("更新失败：" + data.message);
        }
    });
}
</script>
```

后端代码 (Controller):

```
@PutMapping("/api/cart/update")
public Result updateCart(@RequestBody CartItem item) {
    // 业务逻辑：检查库存 -> 更新数据库
    boolean success = cartService.updateCount(item.getId(), item.getCount());
    if (success) {
        return Result.ok(); // 返回成功状态
    } else {
        return Result.fail("库存不足");
    }
}
```

场景三：文章列表“加载更多”（分页追加）

题目描述：

设计一个新闻列表页面。页面底部有一个“加载更多”按钮，点击后加载下一页的新闻并追加到当前列表末尾，不需要刷新页面。

1. 答题话术（设计思路）

> **设计思路：**
>
> 1. **状态管理**：前端维护一个变量 `currentPage`（当前页码），初始值为 1。
> 2. **事件监听**：给“加载更多”按钮绑定 `onclick` 事件。
> 3. **数据请求**：点击时，将 `currentPage + 1`，并发送请求 `/api/news?page=2`。
> 4. **DOM 操作**：
> - 后端返回 JSON 数组。
> - 前端遍历数组，使用模板字符串拼接 HTML，通过 `appendChild` 追加到列表容器底部。
> - 如果返回数据为空，隐藏“加载更多”按钮并提示“到底了”。

2. 具体实现代码

前端代码 (HTML + JS):

```
<ul id="newsList">
    <li>新闻标题1...</li>
</ul>
<button id="loadBtn" onclick="loadMore()">加载更多</button>

<script>
let page = 1; // 当前页码

function loadMore() {
    page++; // 下一页

    fetch(`/api/news?page=${page}&size=10`)
        .then(res => res.json())
        .then(newsList => {
            const list = document.getElementById('newsList');

            if (newsList.length === 0) {
                document.getElementById('loadBtn').innerText = "没有更多了";
                return;
            }

            // 遍历并追加
            newsList.forEach(news => {
                const li = document.createElement('li');
                li.innerText = news.title;
                list.appendChild(li);
            });
        });
}
</script>
```

后端代码 (Controller + MyBatis):

```
// Controller
@GetMapping("/api/news")
public List<News> getNewsList(@RequestParam int page, @RequestParam int size) {
    int start = (page - 1) * size;
    return newsMapper.selectByPage(start, size);
}

// MyBatis XML
// <select id="selectByPage" resultType="News">
//     SELECT * FROM news ORDER BY create_time DESC LIMIT #{start}, #{size}
// </select>
```

💡 总结：综合题“万能公式”

如果考试遇到没见过的场景（比如“点赞”、“评论”、“删除”），请套用以下公式，**必拿高分**：

1. **第一步（UI）**：在哪个元素上，监听什么事件？（`onclick`，`onchange`，`oninput`）。
2. **第二步（Req）**：用 AJAX/Fetch 发什么请求？（GET用于查，POST用于改，参数带什么？）。
3. **第三步（Data）**：后端怎么查库？（Controller 接收参数 -> Mapper 写 SQL）。
4. **第四步（Resp）**：前端拿到数据怎么变？（`innerHTML` 替换，或者 `appendChild` 追加，或者 `alert` 提示）。